ENGINEERING CASE RECORD

# 블링크 개발 문제 해결 기록

눈을 깜빡이면 휴대폰 화면이 움직이면 어떨까 하는 생각에서 시작했습니다. 직접 써 보니 눈이 보이는 것과 의도를 알아듣는 것은 달랐고, 시선이 움직이는 것과 화면의 정확한 위치를 알아내는 것도 달랐습니다. 깜빡임 문법과 특허 검토, Roll·Page·Game의 출시, 보는 힘·흰자·눈꺼풀·얼굴 각도로 이어진 시선 연구를 불편함을 발견하고 가설을 바꿔 간 순서로 정리했습니다. 구현한 것과 검증한 것, 아직 제품 수준에 도달하지 못한 것을 구분한 이력서·면접용 상세 원고입니다.

## 01 프로젝트 정보

| 항목 | 내용 |
| --- | --- |
| 목적 | 필요한 순간 눈동작으로 Android 화면·게임을 조작하는 입력 도구 |
| 기록 기간 | 2026년 6월 18일 초기 기록부터 2026년 10월 15일 연구 기록까지 |
| 담당 범위 | 문제 정의, 아이디어·가설 제안, 실기기 입력과 재현, 우선순위와 채택 판단. 구현·학습·검증·문서화에 AI 코딩 도구 활용 |
| 실행 환경 | Android SM-S·SM-N, Windows 모델 비교, Android·Unity 게임 연결 |
| 주요 기술 | Kotlin, CameraX, MediaPipe, Android 접근성, Rust, Unity/C#, Python·PyTorch/CUDA, 자체 커널과 시간축 모델 |
| 확인된 범위 | Roll·Page·Game 출시 이력. 깜빡임 입력·성능·검증 구조 개선과 별도 시선 연구 |
| 미완료 범위 | 촬영이 바뀌어도 자연스러운 공통 시선 좌표·방향. 시선 Stage 2는 제품 공개와 구분 |
| 특허 상태 | 출원 검토, 명세서와 제출 후보 준비 이력. 접수·등록 완료 증거는 미확인 |
| 문서 기준일 | 2026.10.15 |

공개 포트폴리오 Blink Engineering Case Study

시선 연구 상세 원문 22개 장면의 아이디어와 실험

개발 기간은 매일의 연속 투입 시간을 뜻하지 않습니다. 사용자 관찰과 기록된 실험 날짜를 구분했으며, 이 문서 작성 중 과거 학습이나 실기기 성능 시험을 다시 실행하지 않았습니다. 비공개 원본과 미출원 상세 자료는 싣지 않았습니다.

# 목차

01 프로젝트 정보 . . . 1
02 개요 . . . 3
03 눈은 보이는데 화면은 움직이지 않았다 . . . 3
04 감도 조절에서 눈 입력 문법으로 . . . 3
05 특허를 고민하며 내가 만든 부분을 설명하기 . . . 4
06 앱이 늘면서 드러난 의미와 실행의 경계 . . . 5
07 시선으로 넓히며 흰자와 전체 화면을 생각하다 . . . 6
08 보는 힘과 읽는 줄을 따라가는 도구 . . . 7
09 같은 시기 계속된 문법과 성능 개선 . . . 8
10 각도와 거리 그리고 화면 안팎의 의미 . . . 9
11 다시 쓸 때의 첫 입력과 두 기기의 차이 . . . 10
12 얼굴 자세 추정과 얼굴에서 눈으로 좁히는 모델 . . . 11
13 눈꺼풀과 오차 분해로 가정을 다시 보기 . . . 11
14 아홉 칸에서 축별 방향으로 목표를 줄이기 . . . 12
15 눈가 움직임과 자체 커널 그리고 주시안 . . . 13
16 중립을 찾는 모델과 실제 출시 운영 . . . 14
17 필터와 상태 모델은 체감까지 좋아졌을까 . . . 15
18 처음의 필요와 끝내 남은 문제 . . . 16
19 기술 선택과 확인한 한계 . . . 17
20 프로젝트와 연구 계보 . . . 18
21 이력서에 옮길 때 구분할 표현 . . . 18
22 확인한 원본과 공개 범위 . . . 19

# 02 개요

### 생활 속 입력 도구와 별도의 시선 연구

기록을 읽는 방법

이 기록은 사용자가 기술에 맞춰 눈을 연습하는 대신, 필요한 순간 한 번의 눈동작으로 기기를 쓰게 하려는 과정입니다. 처음에는 깜빡임으로 화면을 움직였고, 이후에는 보는 방향과 읽는 줄까지 입력으로 넓혀 보았습니다.

본문의 아이디어는 실제 사용 중 제가 제기한 관찰과 대화에서 출발합니다. 구현·학습·검증에는 AI 코딩 도구를 사용했습니다. 직접 겪은 불편함, 제안한 가설, 구현된 방법과 실험 결과를 구분해 1인칭으로 재구성했습니다.

여러 연구가 병행됐습니다. 날짜는 확인 가능한 실험 기록을 중심으로 묶었으며 최초 제안일이 남지 않은 흰자 같은 가설에 날짜를 만들어 붙이지 않았습니다. 제품 출시와 연구 후보의 랩·디버그 적용도 별개입니다.

# 03 눈은 보이는데 화면은 움직이지 않았다

2026년 6월 18일

시작은 눈을 깜빡여 화면을 넘기는 프로그램을 만드는 것이었습니다. 영상이나 글을 보는 동안 필요한 동작은 화면 이동인데, 이를 위해 계속 손을 움직여야 했습니다. 손이 닿지 않는 곳에 놓인 휴대폰도 눈동작으로 조작할 수 있다면 쓸 만한 도구가 되겠다고 생각했습니다.

처음에는 브라우저에서 가능성을 확인했습니다. 전면 카메라로 눈을 보고, 깜빡이면 화면을 내리는 실험을 만들었습니다. 그런데 PC에서 열리던 화면을 휴대폰으로 가져가자 카메라 권한 요청부터 나오지 않았습니다. 같은 와이파이에 연결됐다는 사실만으로는 충분하지 않았습니다. 휴대폰에서 카메라를 쓰는 데 필요한 HTTPS 접속 조건을 따로 맞춰야 했습니다.

카메라가 켜진 다음에도 끝이 아니었습니다. 화면에는 제 눈이 보이는데 깜빡여도 스크롤은 내려가지 않았습니다. 눈이 보이는 것, 모델이 눈의 변화를 알아내는 것, 그 결과가 실제 화면 이동으로 이어지는 것은 서로 다른 일이었습니다. 이후 Android 접근성 서비스로 다른 앱의 화면까지 움직이는 경로를 붙이면서, 이 연결을 하나씩 확인하기 시작했습니다.

지금 보면 작은 시작 문제 같지만, 뒤의 개발을 관통하는 질문이 여기서 나왔습니다. 화면에 무언가 표시됐다는 이유만으로 기능이 된다고 말해도 될까. 제가 확인하고 싶은 것은 카메라 미리보기가 아니라, 눈을 한 번 감았다 떴을 때 화면이 정말 한 번 움직이는 일이었습니다.

# 04 감도 조절에서 눈 입력 문법으로

### 내 눈에 맞는 숫자를 찾으면 끝날 줄 알았지만

2026년 6월 말부터 7월 초

실제로 사용하자 이번에는 반대 문제가 나왔습니다. 넘기려 할 때는 반응하지 않는데, 가만히 보거나 얼굴을 움직일 때 화면이 넘어갔습니다. 밝은 곳과 어두운 곳, 가까운 거리와 먼 거리에서 같은 동작의 결과도 달랐습니다.

초기에는 감김 점수, 감은 시간, 감도 단계와 다음 입력까지의 대기 시간을 조절했습니다. 한 상황에 맞는 값을 찾으면 다른 상황이 불편해졌습니다. 오작동을 막으려고 조건을 더하면 정상 입력까지 막혔고, 다시 느슨하게 하면 원하지 않는 이동이 생겼습니다.

더 근본적인 문제도 있었습니다. 사람은 화면을 넘기려고만 눈을 감는 것이 아닙니다. 짧은 영상을 오래 볼 때의 자연스러운 깜빡임과, 웹툰을 빠르게 넘기려는 깜빡임은 겉으로 비슷할 수 있습니다. 감도를 몇 단계 더 만드는 것으로 해결하기 어려운 이유였습니다.

저는 사용자가 앱의 숫자를 외워 눈을 맞추는 방향에 의문을 가졌습니다. 누구는 빠르게 감고 누구는 천천히 감는데, 한 가지 시간으로 같은 뜻을 나눌 수 있을까. 이 질문에서 눈동작 자체의 흐름을 읽는 쪽으로 생각이 옮겨갔습니다.

### 깜빡임을 시간보다 발음과 문법으로 보기 시작했다

2026년 7월 중순

눈을 감는 순간만 잘라 보면 입력이 모호했습니다. 반면 감기기 전부터 다시 뜰 때까지 이어서 보면 모양이 있었습니다. 눈이 닫히고, 정점 부근에서 잠깐 머물 수 있고, 다시 열립니다. 저는 이 흐름을 눈동작의 발음처럼 보기 시작했습니다.

말을 빠르게 하거나 천천히 해도 같은 말을 할 수 있듯이, 깜빡임도 단순히 몇 밀리초를 넘겼는지보다 한 동작 안의 관계가 중요하다고 봤습니다. 닫힘 C, 선택적인 유지 P, 열림 O를 하나의 사건으로 묶었습니다. 일반적인 완성 파형 B와 유지가 있는 파형 H를 구분했고, 이후 H의 유지 관계를 H1과 H2로 나눠 다뤘습니다.

제 요구는 빠른 V형과 느린 V형도 같은 종류의 동작이면 같은 뜻으로 읽어야 한다는 것이었습니다. 전체 시간이 길다는 이유만으로 유지형으로 바꾸면 안 됩니다. 관측 유효성이나 완전 감김을 확인할 기준까지 없애자는 말은 아니었습니다. 원시 신호를 읽는 기준과 완성 동작의 뜻을 덮어쓰는 임시 시간 조건을 구분하려 했습니다.

여기서 문법은 보기 좋은 이름만 붙인 규칙이 아니었습니다. 눈을 다시 떴는지, 실제 유지가 있었는지, 관측이 중간에 끊기지는 않았는지를 함께 확인하는 방법이었습니다. 눈을 찾지 못한 순간을 임의로 채워 "아마 오래 감았을 것"이라고 실행하지 않는 것도 이 설계에 포함했습니다.

발음과 뜻도 나눴습니다. 카메라가 얻은 눈 신호를 같은 기준으로 정리하는 일, 그 신호가 어떤 파형인지 읽는 일, 선택한 사용 방식에서 어느 방향으로 움직일지 결정하는 일을 구별했습니다. 한 동작이 여러 처리 경로를 거쳐도 바깥으로 나가는 행동은 한 번이어야 했습니다.

이때 프로젝트의 중심이 달라졌습니다. 외부 모델로 눈을 찾아 화면을 움직이는 앱에서, 관측한 눈 행동을 일관된 입력으로 해석하는 엔진을 만드는 문제로 바뀌었습니다. 동시에 그 해석 방법을 제 기술로 어떻게 설명할 수 있을지도 고민하게 됐습니다.

다만 파형을 잘 구분한다고 자연 깜빡임과 의도 입력을 항상 알아낸 것은 아닙니다. 두 의도가 같은 완성 파형으로 관측되면 파형만으로는 구별되지 않을 수 있습니다. 이 한계는 나중에 응시와 깜빡임을 결합해 사용 맥락을 더 보자는 질문으로 이어졌습니다.

## 05 특허를 고민하며 내가 만든 부분을 설명하기

### 특허를 고민하며 내가 만든 부분을 다시 설명했다

2026년 7월 중순부터 22일까지 함께 진행한 준비

특허 검토는 7월 초부터 함께 진행했습니다. "눈을 깜빡여 스마트폰을 조작한다"는 소개만으로 충분한지부터 확인해야 했습니다. 관련 선행기술에는 이미 눈 움직임과 깜빡임을 명령에 연결하는 시도들이 있었습니다. 눈으로 조작한다는 큰 아이디어와 제가 발전시킨 구체적인 처리 방법을 나눠 설명할 필요가 있었습니다.

초기 검토는 안전한 스크롤과 실제 사용 피드백을 중심으로 시작했습니다. 이후 파형 문법을 구체화하면서 명세서도 발전시켰습니다. 핵심은 눈 상태를 알아내는 순간 바로 명령을 내리는 것이 아니라, 하나의 완성된 동작을 구성하고 그 관계와 사용 맥락을 해석한 뒤 실행 여부를 결정하는 과정이었습니다.

명세서를 쓰는 일은 개발을 다른 방식으로 되짚는 일이기도 했습니다. 한 번의 깜빡임이 두 번 실행되지 않게 하는 것은 어느 부분의 책임인가. 차단된 입력이 나중에 갑자기 실행되면 어떻게 막을 것인가. 사용자마다 다른 눈의 기준을 반영하면서 같은 동작의 뜻은 어떻게 유지할 것인가. 코드에서는 지나칠 수 있는 질문을 문장과 도면으로 분명히 해야 했습니다.

검토는 아이디어 메모에서 그치지 않았습니다. 여러 초안을 거쳐 명세서와 도면을 준비하고, 공식 작성 도구의 오류 검사와 제출 후보 변환까지 진행한 기록을 남겼습니다. 이미 외부에 보여 준 소개 자료도 따로 정리했습니다. 구체 청구항과 미출원 자료는 이 공개본에 넣지 않았습니다.

그렇다고 특허를 확보했다고 말하지는 않습니다. 확인한 기록의 범위는 출원 검토와 명세서·제출 후보 준비까지입니다. 실제 접수나 등록과는 구분해야 합니다. 제게 중요한 경험은 기능을 만드는 데서 한 걸음 더 나아가, 무엇이 제 설계이고 어떤 문제를 해결하려는 방법인지 끝까지 설명해 본 과정이었습니다.

# 06 앱이 늘면서 드러난 의미와 실행의 경계

## 같은 눈 입력이 책장과 게임으로 이어졌다

2026년 7월 말부터 8월 초

입력의 해석과 행동을 나누자 눈동작을 다른 곳에도 써 볼 수 있었습니다. Roll의 위아래 스크롤뿐 아니라, 악보나 문서를 보는 동안 다음 장으로 넘기는 용도도 생각했습니다. BlinkPage는 같은 눈 입력을 가로 방향 넘김으로 연결하는 별도 앱으로 발전했습니다.

게임도 비슷했습니다. 눈동작이 입력이 될 수 있다면, 화면 이동 외에 게임 행동에도 연결할 수 있었습니다. 다만 Unity 안에 카메라와 깜빡임 판단을 다시 만들면 두 앱이 서로 다른 규칙을 배우게 됩니다. 그래서 공통 엔진이 완성한 사건을 전달하고, 게임은 그 사건을 게임 행동으로 해석하는 연결부를 뒀습니다.

앱이 늘어나니 재사용의 의미도 달라졌습니다. 같은 파일을 복사해서 시작하는 것은 빨랐지만, 한곳의 수정을 다른 곳까지 계속 맞추는 일이 남았습니다. 공통으로 지켜야 할 눈동작 의미와 제품별로 달라야 할 화면 이동을 나누고, 카메라·실행·화면의 책임을 작은 모듈로 정리해 갔습니다.

모든 것을 억지로 하나로 합친 것은 아닙니다. Roll의 수집 경로와 Page·Game의 카메라 경로에는 차이가 있습니다. 공통 규칙을 쓴다는 말만으로 한 앱의 수정이 나머지 앱에도 자동으로 적용됐다고 넘길 수 없었습니다. 각각 어떤 경로를 실제로 쓰는지 확인하는 일이 새 과제가 됐습니다.

## 내 눈을 배울수록 왜 더 이상해질까

2026년 7월 30일부터 8월 7일의 관찰과 재분석

문법을 만들었다고 모든 문제가 사라지지는 않았습니다. 처음에는 되다가 반복할수록 오래 감기가 짧게 감기로 바뀌거나, 앱을 다시 시작하면 느낌이 좋아지는 일이 있었습니다. 사용자의 눈에 맞추려던 기능이 오히려 그다음 입력의 의미를 흔들 수 있었습니다.

기록에서는 앞서 들어온 긴 동작이 일반 깜빡임의 범위를 넓히고, 뒤의 유지형 동작까지 일반형으로 흡수하는 경로가 드러났습니다. 반대로 아주 긴 동작을 먼저 했다는 이유로, 그다음의 분명한 유지 동작을 상대적으로 짧다고 낮춰 보는 문제도 있었습니다.

여기서 개인차를 반영하는 것과 단어의 뜻을 바꾸는 것을 갈랐습니다. 사람마다 다른 열린 눈 크기와 촬영 간격은 비교 가능한 신호를 만드는 데 쓸 수 있습니다. 그러나 앞에서 어떤 동작을 많이 했다는 이유로 다음 완성 파형의 뜻까지 바뀌게 두면 사용자는 규칙을 믿을 수 없습니다.

그래서 현재 동작의 모양과 유지·재열림 관계가 판정을 소유하도록 경계를 정리했습니다. 같은 파형을 순서만 바꿔 넣어도 뜻이 유지되는지 확인했습니다. 사용자에게 잘 맞춘다는 이름의 기능도, 실제로 무엇을 바꿀 권한이 있는지 제한해야 한다는 것을 배웠습니다.

## 고쳤다는 말보다 다시 틀리는 이유가 궁금해졌다

제품 확장 후 반복한 구조 개선과 9월부터 10월의 재검증

실제로 가장 답답했던 순간은 한 문제를 고쳤다고 했는데 다른 곳에서 같은 증상이 다시 나올 때였습니다. 테스트는 통과했다고 하는데 손에 든 휴대폰에서는 안 되는 경험도 반복됐습니다. 수정 횟수나 테스트 개수가 쌓인다는 사실만으로 안심할 수 없었습니다.

살펴보면 실패 입력을 다시 조립하는 코드와 실제 판정기가 서로 다른 재열림 규칙을 쓰는 경우가 있었습니다. 한쪽 시험이 자기 규칙대로 정답을 확인하는 동안 실제 앱의 규칙은 따로 움직인 셈입니다. 이 상태에서는 테스트를 더 붙여도 같은 문제가 다른 연결부에서 나올 수 있었습니다.

이후에는 신고 하나만 통과시키는 데서 멈추지 않으려 했습니다. 실패한 숫자 파형을 재생 자료로 남기고, 사건 번호·시각·세션 순서를 바꿔도 같은 의미를 내는지 검사했습니다. 감지, 파형, 실행 허용, 제스처 접수와 결과를 같은 사건으로 추적해 어디에서 처음 달라졌는지도 나눴습니다.

제가 요구한 구조는 연결부를 하나 둔다는 말로 끝나지 않았습니다. 공통 해석 엔진은 같게 쓰되 Roll·Page·Game의 행동 어댑터와 Debug·R8 스모크·Production의 진단 어댑터를 구분해야 했습니다. 한 앱의 성공이나 공통 파일의 존재만으로 나머지 실행도 같다고 넘기지 말자는 제안이었습니다. 공유 소스, 빌드 설정, 설치 파일과 실제 입력 품질을 각각 비교하도록 검증도 나눴습니다.

코드 분리도 이 목적에 맞춰 진행했습니다. 파일을 작게 만드는 것보다 누가 판단을 소유하고 누가 결과를 실행하는지를 분명히 하는 것이 중요했습니다. 문제·원인·수정·검증·남은 위험을 연결한 원장은, 다음에 같은 증상이 나왔을 때 다시 처음부터 짐작하지 않기 위한 기록이 됐습니다.

접근성이 여러 개 켜져 있다는 이유로 다른 서비스를 모두 꺼서 통과시키는 시험에도 반대했습니다. 접근성 설정이 켜진 상태와 사용자가 실제 감지를 시작한 상태는 다릅니다. 입력 중인 세션만 관측과 실행을 소유하고, 멈춘 세션이나 이전 실행의 늦은 결과는 행동을 만들지 않아야 했습니다. 소유자와 실행 세대를 검사하는 구조를 유지했지만 모든 제조사·설치 조합을 검증한 것은 아닙니다.

# 07 시선으로 넓히며 흰자와 전체 화면을 생각하다

## 깜빡임 다음에는 보는 곳을 입력으로 만들고 싶었다

시선 연구 초기부터 2026년 8월 기록

깜빡임으로 한 번씩 명령할 수 있게 되자, 눈을 감지 않은 동안의 움직임도 쓰고 싶어졌습니다. 게임에서 왼쪽을 보면 왼쪽으로 이동하고, 글을 읽을 때는 읽는 줄을 표시하면 어떨까. 눈을 단순한 버튼에서 방향을 가진 입력 장치로 넓혀 보려는 생각이었습니다.

처음에는 화면에 작은 표적을 띄우고 따라보았습니다. 그런데 표적 하나를 똑바로 보는 시험과, 하늘에서 떨어지는 물체를 보며 피하는 게임은 달랐습니다. 게임에서는 화면 전체를 살피며 다음 행동을 정합니다. 표적을 맞힌 기록만 높다고 실제 조작도 편하다고 볼 수 없었습니다.

MediaPipe로 얼굴과 눈의 위치를 찾는 부분은 활용하되, 그 결과를 화면의 어느 위치로 바꿀지는 자체 모델과 제어기로 연구했습니다. 외부 모델을 쓴 부분, 제가 제안한 가설, AI 코딩 도구와 함께 만든 구현, 직접 눈으로 확인한 결과를 한 덩어리의 성과로 쓰지 않으려 했습니다.

## 흰 화면에서 잘된다면 눈동자 말고 흰자를 보면 어떨까

초기 아이디어 제안일 미확인과 2026년 8월 조도 실험

밝은 웹툰 화면에서는 인식이 되는 경험이 있었습니다. 화면이 흰색이나 밝은색이면 눈에 반사돼 눈의 모양을 더 쉽게 구별하는 것은 아닐까 생각했습니다. 화면을 단순히 결과를 보여 주는 곳이 아니라, 눈을 관찰하는 빛의 조건으로도 보자는 제안이었습니다.

별도의 갈래로 흰자를 보자는 아이디어도 제안했습니다. 작고 어두운 눈동자의 중심만 찾기 어렵다면, 눈동자가 움직일 때 달라지는 흰자의 모양과 양을 비교하면 어떨까. 밝은 화면은 관찰 조건을 바꾸는 생각이고, 흰자는 읽어 낼 단서 자체를 바꾸는 생각이었습니다. 이 제안의 최초 날짜와 흰자 잔차 단독 후보의 완결된 성적표는 현재 연결한 기록만으로 확정하지 않았습니다.

후속 실험에서는 어두운 화면·흰 화면·민트 화면의 조건을 나누고, 주변 조도와 실제 모델에 들어간 눈 영상의 밝기도 따로 남겼습니다. 밝은 화면이 언제나 유리하다는 결과는 아니었습니다. 세 조건을 모두 갖춘 초기 관측도 부족했고, 그중 한 회는 오히려 어두운 조건의 오차가 더 낮았습니다.

흰자라는 아이디어도 버리지 않았지만, 밝은 픽셀을 모두 흰자라고 부를 수는 없었습니다. 피부와 반사광도 밝게 보입니다. 나중에는 눈 안쪽의 명암 분포와 경계를 보존하는 입력까지 비교했으나, 흰자 영역의 정답을 따로 붙인 분할 모델을 완성했다고 쓰지는 않습니다. 이 경험은 단서를 제안하는 일과 그 단서를 실제로 측정했다는 증명이 다르다는 것을 알려줬습니다.

### 가운데만 잘 맞는 시험으로 화면 전체를 설명할 수 없었다

2026년 8월 전체 화면 측정과 테두리 보정

가운데 몇 칸만 보는 시험으로는 화면 모서리에서 되는지 알 수 없었습니다. 저는 기기 전체 화면을 따라 한 바퀴 도는 귀여운 표적을 두고, 사용자가 그것을 따라보면 어떻겠느냐고 제안했습니다. 현재 손에 든 휴대폰의 거리와 각도에서 화면의 끝이 어디인지 먼저 알려 주려는 생각이었습니다.

실제로 보정 뒤 모드 선택의 위·가운데·아래 이동이 자연스러워졌다는 체감이 있었습니다. 그러나 게임을 하고 돌아오면 보정이 풀린 듯하거나, 표적이 화면 끝까지 가지 않거나, 범위 부족 안내가 같은 곳에서 반복되는 문제가 나왔습니다. 보정을 완료했다고 표시하는 것과 이후에도 잘 맞는 것은 달랐습니다. 나중에는 테두리만 천천히 도는 대신 전체 화면을 빠르게 따라보며 수집하자는 방법도 제안했습니다.

그래서 표적의 전체 화면 범위와 실제 이동 경로, 눈을 감아 표본을 못 받은 시간, 정답 표적을 본 뒤 어느 시각의 눈을 평가했는지를 나눠 기록했습니다. 측정이 너무 길면 집중이 흐려진다는 의견에 따라 짧은 반복으로 바꾸는 작업도 이어졌습니다. 숨을 참아야 통과할 만큼 엄격한 측정은 자연스러운 사용을 설명하지 못했습니다.

8월 16일의 한 보정 비교는 평균 좌표 오차를 0.225812에서 0.213343으로 줄였지만, 다른 회귀 비교의 3×3 구분은 46.43%에서 42.86%로 떨어졌습니다. 같은 지표의 전후가 아닌 두 검사 결과입니다. 보정이라는 절차 자체를 성공으로 세지 않고, 좌표와 구역 구분이 각각 좋아지는지 확인해야 했습니다. 이후에는 매번 등록해야만 쓰는 방식 대신 공통 모델 자체를 개선하는 쪽에 무게를 뒀습니다.

## 08 보는 힘과 읽는 줄을 따라가는 도구

### 마우스처럼 보는 힘으로 포인터를 움직이면 어떨까

2026년 8월 22일부터 23일

시선 점이 들쭉날쭉하게 튀자 마우스가 떠올랐습니다. 마우스 포인터는 손이 가리키는 절대 위치에 순간이동하지 않습니다. 손으로 힘을 줘 움직이고 멈춥니다. 시선도 현재 보는 위치를 그대로 찍는 대신, 이미 있는 포인터를 밀어 움직이는 힘으로 쓰면 어떨까 생각했습니다.

이것이 보는 힘 실험의 출발이었습니다. 직접 좌표와 힘을 거친 포인터를 같은 입력으로 비교했습니다. 8월 22일 자유 조향 두 회에서 프레임 사이 이동 중앙값은 약 75~79% 작아졌고, 수평 방향 반전률도 21.6~26.2%에서 4.7~4.9%로 줄었습니다. 다만 이 시험에는 정답 표적이 없었습니다. 덜 흔들렸다는 뜻이지, 보는 곳을 그만큼 정확히 알아냈다는 뜻은 아니었습니다.

속도로 해석한 방식에는 새 문제가 있었습니다. 표적을 계속 보고 있으면 포인터가 도착한 뒤에도 같은 방향으로 달아났습니다. 그래서 랩에서는 좌표 쪽으로 탄성 있게 따라가되 속도를 제한하는 후보도 비교했습니다. 두 세션으로 설정을 고르고 남겨 둔 세 번째 세션에서 속도형 대비 중앙 2D 오차가 0.590에서 0.288로 줄었습니다. 직접 좌표와의 오차는 거의 같고 이동량은 13.4% 적었습니다.

여기서 제품의 공통 힘 제어와 랩의 탄성 추종 후보를 구분했습니다. 서로 다른 방법을 모두 같은 보는 힘이라는 이름으로 묶으면 무엇이 나아졌는지 알 수 없기 때문입니다. 이후 질문도 바뀌었습니다. 포인터를 편하게 움직이게 하는 문제와, 처음부터 올바른 방향을 알아내는 문제를 따로 풀어야 했습니다.

### 좌우는 되는데 아래로 내려오지 않는 이유를 나눴다

2026년 8월 22일부터 이어진 상하 비교

직접 써 본 느낌은 좌우는 생각보다 쓸 만한데 상하가 잘 안 된다는 것이었습니다. 한 번 표적을 놓치면 움직이는 방향은 비슷해도 실제 좌표는 다시 맞지 않았습니다. 여기서 힘을 더 세게 주면 해결될지, 원래 상하 신호부터 부족한지 구분해야 했습니다.

같은 날의 한 측정에서 포인터의 세로 범위는 화면 비율 약 0.405~0.639에 머물렀습니다. 화면 전체를 움직이기에는 너무 좁았습니다. 그렇다고 상하 정보가 전혀 없는 것도 아니었습니다. 같은 입력의 표적 방향 상관은 수평 0.883, 수직 0.698로 달랐고, 별도 얼굴 수직 신호에는 더 뚜렷한 변화가 남은 기록도 있었습니다.

수직만 속도로 누적하거나 얼굴 신호를 사용하고, 중앙 기준과 문맥을 달리하는 후보가 이어졌습니다. 움직임 범위를 넓히는 데 성공해도 잘못된 방향이나 누적 오차까지 고친 것은 아니었습니다. 보정이 세 세션 모두 적용된 후속 후보도 실제 오차 개선은 한 세션에서만 나타났습니다.

이 과정에서 아래보기와 눈 감김이 닮아 보인다는 문제가 계속 남았습니다. 전면 카메라는 눈보다 위에 있으니 아래를 볼수록 눈동자가 눈꺼풀에 가려질 수 있겠다는 제 관찰도 이후 눈꺼풀 실험으로 이어졌습니다. 아직 검증하지 않은 관찰을 그때의 확정 원인으로 소급하지는 않았습니다.

### 글을 읽는 순서를 외우는 것은 시선 추적이 아니었다

2026년 8월 29일부터 31일

시선으로 게임만 하는 것이 아니라 지금 읽는 줄을 표시하는 BlinkLine도 시도했습니다. 그런데 사람이 오른쪽 끝을 보면 다음 줄 왼쪽으로 옮겨 주는 규칙을 넣으면 그럴듯하게 보일 수 있습니다. 저는 읽다가 다시 맨 위를 보면 그 줄로 따라와야 한다고 요구했습니다.

이전 줄로 되돌아보거나 중간 문장을 다시 읽는 일도 실제 독서입니다. 정해 둔 독서 순서와 임계값으로 줄을 넘기는 것을 보는 위치를 알아냈다고 부를 수 없었습니다. 그래서 미리 정한 줄바꿈 문법을 제거하고, 실제 화면에 그려진 줄의 위치와 현재 시선 관측을 연결하는 방향으로 바꿨습니다.

여기서 모델 밖의 버그도 드러났습니다. 시선 결과가 늦게 도착하거나 얼굴을 다시 찾은 뒤 예전 좌표가 살아나면, 줄 추적기는 틀린 위치를 계속 따라갈 수 있습니다. 카메라 관측, 유효한 축, 새 관측 세대, 실제 줄 배치를 각각 분리했습니다.

이것으로 자연스러운 독서 추적을 완성한 것은 아닙니다. 위쪽으로 돌아갈 때 지연되거나 세로 원신호가 부족한 문제가 남았습니다. 다만 줄 선택 규칙을 바꾸어 점수를 꾸미는 것과, 실제 시선을 더 잘 알아내는 일을 구별할 기준은 분명해졌습니다.

## 09 같은 시기 계속된 문법과 성능 개선

### 문법으로 풀겠다면서 다시 숫자를 붙일 수는 없었다

2026년 8월 말부터 9월 초

양방향 이동에서는 짧게 감아 기본 방향으로 가려 했는데 반대로 움직이는 신고가 나왔습니다. 이를 막기 위해 실행 직전에 별도의 최소 감김 시간을 요구하는 수정도 등장했습니다. 당장 반대 이동을 줄일 수는 있지만, 그러면 문법을 만들겠다는 처음의 생각에서 다시 멀어집니다.

저는 이 방식을 임시방편으로 봤습니다. 유지가 있는지 판단하는 부분이 잘못됐는데 행동을 내보내는 마지막 문에 시간 자물쇠를 더 거는 것은 다른 문제입니다. 결국 같은 길이의 동작이라도 실제 정점 부근에 유지가 있는지, 감김 점수가 먼저 포화돼 가짜 유지처럼 보이는지 확인해야 했습니다.

별도 시간 조건은 폐기하고, 파형 안의 유지 관계를 다시 확인하도록 고쳤습니다. 같은 292ms와 같은 관측 수를 가진 입력도 정점이 기울어진 경우와 평평하게 유지된 경우를 구분하는 회귀 검사를 남겼습니다. 이후 완전히 감겼다가 다시 열렸는지의 확인도 양방향 명령 경계에서 함께 다뤘습니다.

이 과정은 규칙을 많이 붙이는 것이 구조 개선은 아니라는 사실을 보여 줬습니다. 무엇을 놓쳤는지 모르는 채 마지막 단계에서 막으면 한 문제는 잠잠해져도 정상 입력까지 같이 잃을 수 있었습니다. 문법을 쓰겠다는 생각은 구현 중에도 반복해서 지켜야 하는 기준이었습니다.

### 잘 알아보려면 더 봐야 하는데 많이 보면 느려졌다

2026년 9월 성능 조사

입력을 놓치지 않으려면 눈동작의 중요한 순간을 충분히 봐야 했습니다. 그런데 릴스와 함께 사용하면 버벅임과 발열이 생겼습니다. 덜 계산하면 반응이 나빠지고 더 계산하면 휴대폰 사용이 불편해지는 충돌이었습니다. 저는 성능을 조절하더라도 인식 품질은 먼저 지켜야 한다고 봤습니다. 쓰려고 만든 도구가 입력부터 놓치면 낮은 전력 사용량도 의미가 작기 때문입니다.

저는 열이 난다는 진단만으로 입력을 막거나, 인식 품질을 잃으면서 전력 수치만 낮추는 방향에도 반대했습니다. GPU를 쓰는 것과 불필요한 작업을 쌓지 않는 것은 다른 선택입니다. 일부 수정에서 핵심 입력을 발열 관측값만으로 중지하지 않도록 했지만, 그것을 발열이 해결됐다는 뜻으로 쓰지는 않습니다.

GPU가 켜져 있는지만 확인해서는 답이 나오지 않았습니다. 영상 복사와 변환, 대기 중인 작업도 따로 비용을 만들었습니다. 카메라는 새 프레임을 계속 보내는데 모델은 그 속도로 끝내지 못해 오래된 일이 쌓이는 경로를 찾았습니다.

Roll에서는 작업을 복사하기 전부터 대기·실행이 끝날 때까지 한 예산으로 관리하도록 바꿨습니다. 실행 하나와 후속 하나만 미완료로 둘 수 있게 했습니다. 같은 비용을 준 재현 시험에서 최대 대기는 503ms에서 67ms로 줄었습니다. 이는 실제 릴스 전체의 속도 개선율이 아니라, 오래된 작업이 쌓이는 구조를 바꾼 결과였습니다.

입력 품질을 깎지 않고 줄일 계산도 찾았습니다. Rust 영상 변환에서는 크기가 같은 회전 변환의 불필요한 보간 계산을 줄이고, 크기를 바꾸는 경로는 그대로 뒀습니다. 1,388개 합성 조건에서 결과가 같은지 비교한 뒤 SM-S의 해당 변환 시간이 1.7013ms에서 0.6482ms로 줄었음을 확인했습니다.

이렇게 성능 개선을 두 갈래로 나눴습니다. 같은 결과를 더 적은 계산으로 만드는 일과, 감당할 수 없는 작업을 무작정 쌓지 않는 일입니다. 둘 다 필요했지만 어느 쪽도 구형 기기의 모든 인식 실패와 발열을 해결했다고 확대하지 않았습니다.

# 10 각도와 거리 그리고 화면 안팎의 의미

## 휴대폰 각도와 거리를 모델 밖의 핑계로 두지 않았다

2026년 9월 3일 자세 계측과 이후 거리 관찰

손에 든 휴대폰은 고정된 실험 장비가 아닙니다. 같은 곳을 봐도 기울기와 거리가 달라지면 눈이 다르게 보입니다. 그래서 랩에서 휴대폰 각도도 함께 모으자고 제안했습니다. 9월 3일 기록에서는 기기 자세를 관측에 연결하는 계측을 확인했지만, 알려진 자세 안의 작은 이득이 보지 못한 자세까지 그대로 이어지지는 않았습니다.

제게 특히 어려웠던 것은 약 -59도로 표시된 자세에서 아래를 보는 일이었습니다. 저는 나중에 그 이유가 각도 하나가 아니라 휴대폰이 너무 가까웠기 때문일 수도 있다고 가정했습니다. 이 값은 당시 표시된 기기 각도이지 눈이나 얼굴의 실제 각도를 뜻하지 않습니다. 가까움과 기울어짐이 함께 바뀐 경험만으로 원인을 하나로 확정할 수는 없었습니다.

거리를 알려면 새 센서를 달아야 하는지도 고민했습니다. 하지만 비용과 기기 제약을 늘리고 싶지는 않았습니다. 그래서 카메라 안에서 눈이 차지하는 상대 크기와 두 눈 사이 간격 같은 값으로 먼저 비교하기로 했습니다. 눈이 작게 찍혔다는 대리 지표이지 몇 cm 떨어졌는지 재는 자는 아니었습니다.

멀리, 더 멀리, 손으로 든 상태와 고정한 상태로 반복하며 조건을 나눴습니다. 눈이 안 보이면 측정을 전부 멈추는 방식에 대해서도 의문을 제기했습니다. 안 보이는 상황 자체를 실패 관측으로 남겨야 어느 조건에서 도구가 멈추는지 알 수 있기 때문입니다. 다만 없는 눈 영상을 만들어 정상 입력으로 학습시키자는 뜻은 아니었습니다.

## 화면 밖을 보고 깜빡이면 다른 뜻을 주면 어떨까

2026년 9월 시선과 깜빡임 결합 실험

시선은 좌표만을 위한 정보가 아닐 수 있었습니다. 같은 깜빡임이라도 화면 안을 보고 한 것과 화면 밖을 보고 한 것을 다르게 해석하면 오작동을 줄일 수 있지 않을까 생각했습니다. 카메라 위쪽을 보고 깜빡이면 일시정지하고, 다시 같은 동작을 하면 재개하는 식의 제안이었습니다.

처음에는 화면 상단에 집중하는 단계부터 잘 넘어가지 않았습니다. 상단을 더 올려 카메라와 화면 밖을 기준으로 삼자는 제안, 눈만 올린 경우와 얼굴 전체를 움직인 경우를 나누자는 제안으로 이어졌습니다. 화면 안과 밖을 번갈아 모으는 수집 흐름도 사용자가 이해할 수 있는지 다시 봤습니다.

등록 뒤에만 동작한다면 모든 사용자가 매번 등록해야 하는 것 아니냐는 문제도 제기했습니다. 저는 공통 모델을 원했지, 개인 보정을 통과해야만 쓸 수 있는 제품을 원한 것은 아니었습니다. 초기 테두리 보정은 가능성을 보는 방법이었지만, 나중에는 그것을 사용자에게 필수 절차로 맡기지 말자는 방향으로 바뀐 것입니다. 수집과 학습은 랩으로 빼고, 실제 사용 앱은 검증된 모델을 받아 실행하도록 경계를 나누는 방향을 요구했습니다.

화면을 보고 있는데도 다시 화면을 보라는 안내가 나오는 경우에는 애매한 관측과 화면 밖을 본 판단을 같은 상태처럼 쓰는지 물었습니다. 정지 영역도 사용자가 알 수 있어야 했고, 실행되면 화면에 크게 나타났다가 사라지는 피드백을 요구했습니다. 해당 외부 시험에서 정지·재개한 것은 눈 입력이었으며, 영상 앱의 재생 버튼을 직접 제어한 기능으로 쓰지는 않습니다.

이 실험은 시선 좌표와 목적이 달랐습니다. 화면 안과 밖을 구별할 수 있다고 3×3이나 정확한 좌표까지 아는 것은 아닙니다. 반대로 좌표 연구에서 모은 화면 안 기록은 안쪽의 예로 재활용할 수 있지만, 화면 밖의 정답까지 생기는 것은 아닙니다. 이 가지는 별도 실험으로 남겼고 깜빡임 제품의 기본 문법과 섞지 않았습니다.

# 11 다시 쓸 때의 첫 입력과 두 기기의 차이

## 한참 안 쓰다가 다시 쓸 때도 도구여야 했다

2026년 10월 초

연속해서 깜빡일 때 잘되는 것만으로는 충분하지 않았습니다. 영상을 끝까지 보고 넘기거나, 다른 곳을 봤다가 돌아오거나, 잠금을 풀고 다시 쓰는 순간에도 첫 입력이 중요했습니다. 자동 절전을 껐는데 몇 시간 뒤 꺼져 있다는 신고도 이 맥락에 있었습니다. 마우스나 키보드처럼 필요할 때 바로 쓸 수 있어야 한다는 것이 제 기준이었습니다. 얼굴이 잠깐 사라졌다는 이유로 입력을 영구 종료하는 것과 배터리를 위한 자동 절전 옵션, Android가 프로세스를 종료하는 상황도 나눠 봐야 했습니다.

10월 Roll 조사에서는 별도의 잠금 종료 제한과 세션 소유권 문제가 드러났습니다. 외부 입력이 잠금 해제 후 재개될 예정인데, 메인 연습 화면은 지금 카메라가 돌지 않는다는 이유로 빈자리로 볼 수 있었습니다. 연습 화면이 카메라를 먼저 가져가면 사용자가 하던 외부 제어는 끊깁니다.

Roll에서는 지금 실행 중인 상태뿐 아니라 준비·재개 예약도 소유권으로 관리했습니다. 잠금 시간만으로 끝내는 제한을 제거하고, 다시 시작하는 재시도 정책은 따로 뒀습니다. 수정 뒤 12분 잠금과 해제 후 재개를 실기기에서 확인했습니다. Page는 별도 활성 상태 정책을 이미 사용했으므로 이 Roll 수정을 모든 앱에 새로 적용한 것처럼 합산하지 않았습니다. Android가 프로세스를 종료한 뒤의 복구까지 같은 시험으로 증명한 것도 아닙니다.

이런 경험을 거치며 일시정지, 종료, 재개 예약이 화면에 비슷하게 보여도 내부에서는 다른 상태여야 한다는 것을 배웠습니다. 앱을 켜 놓고 계속 시험하는 개발 환경보다, 필요할 때 한 번씩 사용하는 생활 속 흐름이 더 어려운 시험이었습니다.

## 오래된 휴대폰이라서 그렇다고 끝내지 않기로 했다

2026년 10월 7일 기기 비교

SM-S에서는 되는데 SM-N에서는 오래 감기가 안 되는 경우가 남았습니다. 구형 기기는 처리 능력이 낮으니 어쩔 수 없다고 보기 쉬웠습니다. 실제로 처리 시간 차이는 있었지만, 감김 표시가 나오는데 액션이 없는 문제까지 그 말 하나로 설명할 수 있는지는 따로 봐야 했습니다.

사용자 동의를 받아 짧은 입력 기록을 남기고, 정상 SM-S 입력과 실패 SM-N 입력의 같은 픽셀을 양쪽 기기의 CPU·GPU에 다시 넣었습니다. 기록기 자체가 입력 크기 변경을 잘못 처리한 오류도 먼저 발견해 고쳤습니다. 제대로 모으지 못한 기록을 정상 시험 결과라고 세지 않기 위해서였습니다.

46프레임 교차 재생에서는 실패 입력을 빠른 기기로 옮겨도 한쪽 눈의 감김 관측이 부족하게 남았습니다. 반대로 SM-S의 정상 입력은 SM-N에서도 낮은 눈 개방도로 관측됐습니다. 해당 실패를 느린 스크롤 실행부 탓으로만 돌릴 근거는 약해졌습니다.

기기를 검사하는 도구가 시험을 바꾸는 문제도 발견했습니다. 화면 구조를 읽는 기본 UI 자동화 명령이 접근성 서비스를 잠시 억제해 감지를 끊었습니다. 설정 목록에는 켜져 있어도 실제 연결은 달라질 수 있었습니다. 그 구간을 정상 연속 동작의 증거로 세지 않고, 해당 조회를 중단한 뒤 화면 캡처·로그·서비스 상태로 재확인했습니다. 검사 때문에 만든 실패와 실제 사용 중의 실패를 구분해야 했습니다.

조사 범위는 입력 영상과 눈 관측 단계로 좁혀졌지만, 실제 눈 상태·촬영 조건·모델 오차 중 무엇이 얼마만큼 영향을 줬는지는 남았습니다. 실패를 통과시키려고 감김 기준을 낮추지는 않았습니다. 원인을 좁힌 것과 문제를 해결한 것은 여기서도 다른 성과였습니다.

# 12 얼굴 자세 추정과 얼굴에서 눈으로 좁히는 모델

## 얼굴 각도를 배우게 하자고 했는데 기록에는 각도가 없었다

2026년 10월 8일

촬영을 바꾸면 잘 안 맞는 문제가 남자 저는 학습할 때 얼굴 각도도 배우는지 물었습니다. 눈의 같은 모양도 얼굴이 기울어 있으면 다른 뜻일 수 있으니, 눈 영상만 잘라서 보는 가정부터 확인할 필요가 있었습니다.

실제로 기록을 열어 보자 과거 좌표 자료 947행에 얼굴 자세 추정값이 연결된 행은 0개였습니다. 얼굴 변환행렬이 특정 자세 시험에서만 요청되고 좌표 시험에서는 빠져 있었습니다. 휴대폰 각도나 눈 사각형의 비대칭 값이 있다고 사람 얼굴의 좌우·상하·회전 각도도 얻은 것은 아니었습니다.

좌표 측정에서도 얼굴 변환행렬을 요청하고 그로부터 계산한 각도 추정값, 누락, 관측 시각을 구분해 저장하도록 바꿨습니다. 새 실측 346행에는 모두 이 추정값이 연결됐습니다. MediaPipe의 관측으로 계산한 값이지 외부 장비로 잰 물리적 정답은 아닙니다. 뒤의 프레임에 있는 각도를 앞선 눈 영상에 가져다 붙이거나 오래된 마지막 값을 현재 측정처럼 쓰지 않도록 시간 경계도 확인했습니다.

여기서 얻은 것은 정확도 상승보다 먼저 제대로 된 비교 조건이었습니다. 눈과 각도를 넣은 후보가 일부 오차를 줄이더라도 정밀 적중이나 다른 촬영에서 나빠지면 교체할 수 없었습니다. 얼굴 각도를 추가했다는 구현 사실과 자세에 강해졌다는 결과는 따로 확인해야 했습니다.

## 눈에서 얼굴로 넓히는 대신 얼굴에서 눈으로 좁혀 보았다

2026년 10월 8일

앞서 시선과 다른 조작 방법으로 얼굴을 좌우로 돌리면 에임도 움직이게 하자는 제안이 있었습니다. 이것은 머리 움직임 입력이지 시선 좌표를 찾은 성과가 아닙니다. 이후에는 눈동자를 하나의 점, 얼굴을 그 점의 확장으로 보자는 가정을 제시했습니다. 다시 순서를 뒤집어 얼굴로 대략적인 위치를 추론한 뒤 눈으로 정확한 위치를 찾자는 생각으로 발전했습니다. 아래 날짜는 이 후속 비교의 기록일이며 선행 아이디어의 최초 제안일은 아닙니다.

이를 같은 프레임에서 눈만 보는 모델, 얼굴만 보는 모델, 둘을 병렬로 합친 모델, 얼굴의 첫 추정을 눈이 수정하는 모델로 비교했습니다. 얼굴과 눈의 저장 시각이 다르면 같은 순간의 관측으로 볼 수 없어 정확히 일치하는 학습 45행과 평가 55행을 사용했습니다. 자료 수가 줄어든 이유도 남겼습니다.

얼굴에서 시작해 눈으로 수정하는 학습 자체는 움직였습니다. 그러나 얼굴의 첫 추정이 충분히 정확한 기준이 되지는 못했습니다. 반복 촬영에서는 세로 평균 오차가 눈 전용 대비 약 12.10% 줄어도 전체 2D 오차는 약 2.89% 커졌습니다. 작은 눈 조건에서는 반대로 2D 오차가 약 11.22% 줄고 세로 오차는 약 4.32% 커졌습니다.

얼굴을 먼저 따로 학습해 고정한 뒤 눈이 수정하게 하는 후속도 비교했지만 일관된 개선은 얻지 못했습니다. 얼굴을 크게 보면 정보가 많아질 수 있다는 가설과, 현재 자료·학습 방식으로 유용한 첫 기준을 만들었다는 증명은 달랐습니다. 이 결과로 얼굴 단서 전체가 쓸모없다고 결론 내리지는 않았습니다.

# 13 눈꺼풀과 오차 분해로 가정을 다시 보기

## 눈동자가 가려진다면 눈꺼풀 모양만으로 상하를 보면 어떨까

2026년 10월 8일

아래를 볼 때 전면 카메라에는 눈꺼풀만 더 잘 보이는 것 같았습니다. 그렇다면 눈동자를 끝까지 찾으려 하지 말고, 눈꺼풀이 내려오는 모양으로 상하를 판단하면 어떨까. 잘 보이지 않는 부분을 억지로 복구하는 대신 보이는 부분의 변화를 쓰자는 제 아이디어였습니다.

같은 자료에서 눈 양 끝점을 기준으로 위·아래 눈꺼풀의 윤곽을 정리하고, 그 모양으로 세로 위치를 학습하는 작은 모델을 비교했습니다. 단순히 눈이 몇 픽셀 이상 열렸으면 위라고 정한 것이 아니라, 좌표를 예측하도록 학습했습니다. 다만 윤곽을 찾는 앞단은 기존 얼굴 검출기를 사용했습니다.

반복 촬영의 상·중·하 구분은 기존 눈 CNN의 79.861%에서 87.500%로 올랐습니다. 특히 아래 구분은 좋아졌습니다. 그런데 작은 눈·자세 변경 촬영에서는 같은 후보가 55.556%로 떨어졌고, 두 조건의 연속 세로 좌표 오차도 기존 눈 모델보다 나빴습니다.

그래서 눈꺼풀만으로 교체하지 않았습니다. 상하 구간을 나누는 데 도움이 된 관측과, 정확한 세로 좌표를 얻는 일은 다르기 때문입니다. 이 자료에는 실제 깜빡임과 아래보기를 따로 확인한 정답도 없었습니다. 아래보기 단서를 시험했다는 사실을 깜빡임 오탐까지 해결한 성과로 바꿔 쓰지 않았습니다.

### 흔들림을 없애도 틀린 곳을 보면 여전히 틀렸다

2026년 10월 8일 좌표 오차 분해

계속 모델을 바꾸기 전에 지금 오차가 어떤 모양인지 나눠 보았습니다. 같은 곳을 보는데 점이 흔들리는지, 일정하게 엉뚱한 곳을 가리키는지, 촬영을 바꿀 때 기준이 달라지는지는 서로 다른 문제였습니다.

두 촬영 344평가행의 한 대조 모델에서 세로 제곱오차를 나누자 방문 안의 흔들림 몫은 4.196%, 공통 위치 편향은 71.059%, 촬영 간 불일치는 24.745%였습니다. 이는 그 자료를 수학적으로 나눈 결과이지, 생리적 원인이나 거리·각도의 영향 비율을 알아냈다는 뜻은 아닙니다.

같은 계산에서 흔들림만 이상적으로 전부 없애도 세로 RMSE는 화면 높이 대비 9.542%에서 9.340%로 조금 줄었습니다. 미래 프레임까지 사용하는 진단 계산이므로 실제 필터 성능도 아닙니다. 하지만 그 자료에서는 부드럽게 만드는 작업만으로 큰 위치 오차를 해결하기 어렵다는 점을 확인할 수 있었습니다.

보는 힘 실험에서 느낀 한계를 다른 방법으로 다시 확인한 셈입니다. 흔들림 억제, 잘못된 위치의 수정, 촬영이 바뀌었을 때의 전달을 나눠 보지 않으면 같은 부드러움만 계속 개선하면서 좌표도 나아졌다고 착각할 수 있었습니다.

## 14 아홉 칸에서 축별 방향으로 목표를 줄이기

### 정밀 좌표에서 아홉 칸으로 다시 좌우와 상하로 목표를 줄였다

고정 조건을 우선한 선행 결정과 2026년 10월 9일부터 10일의 범위 축소

범위를 줄이기 전에도 먼저 폰과 사용자를 고정해 기본 기능이 되는지 보자고 했습니다. 흔들림과 먼 거리까지 한꺼번에 해결하려다 무엇이 안 되는지 놓치지 않기 위해서였습니다. 랩에서 고정 조건의 정확도를 확인한 뒤 게임에 넣자는 순서였고, 고정 조건이 곧 일상 사용 전체를 대표한다는 주장은 아니었습니다.

정확한 화면 좌표를 계속 요구해도 만족스러운 결과가 나오지 않자, 먼저 성공 범위를 제한하자고 했습니다. 화면을 3×3으로 나눈 아홉 구역까지만 구별하고, 그것도 어렵다면 좌·중립·우와 상·중립·하를 따로 개발하자는 방향이었습니다.

고정 조건의 10월 8일 진단에서도 학습한 입력을 다시 맞히는 것과 뒤의 입력을 맞히는 것은 달랐습니다. 한 촬영의 증강 없는 재대입 정밀 적중은 95.68%였지만 같은 방문의 뒤쪽 프레임 평가는 29.94%였습니다. 다른 촬영에서만 틀리는 문제라고 좁힐 수도 없었습니다. 같은 방문을 공유한 비교이므로 독립 제품 정확도로 보지는 않았습니다.

여기서 가운데는 중요했습니다. 좌우만 번갈아 맞히는 모델이라면 사용자가 가운데를 볼 때도 캐릭터가 움직일 수 있습니다. 가운데에서 가만히 있어야 할 때와 방향을 바꾸려 할 때를 나눠 평가했습니다. 게임에서는 당장 필요한 RUN의 좌우 조작에 집중하고 다른 모드는 삭제하지 않은 채 개발을 동결하는 방향으로 범위를 더 좁혔습니다.

축을 나누면 한쪽을 바꾸면서 다른 쪽을 보존하는 시험이 가능했습니다. 한 상하 후보는 근거리 구분이 84.131%에서 85.119%로 오르고 중앙 오판이 23.123%에서 3.709%로 줄었습니다. 그러나 아래 재현율과 다른 촬영의 퇴행이 남아 그 결과만으로 제품 교체를 결정하지 않았습니다.

좌우와 상하의 점수가 있다고 두 값을 평균해 3×3 정확도라고 부르지도 않았습니다. 아홉 칸은 두 방향이 동시에 맞아야 합니다. 범위를 줄인 것은 실패를 성공으로 이름 바꾼 것이 아니라, 어느 부분부터 실제로 쓸 만하게 만들지 정한 결정이었습니다.

### CNN을 바꾸기 전에 실제 눈이 몇 픽셀인지 확인했다

2026년 10월 12일 RUN 원인 조사와 해상도 비교

게임에서 좌우가 불편하자 Unity 때문에 늦는지, 모델이 방향부터 잘못 읽는지 다시 나눴습니다. 같은 입력을 랩과 게임의 전달 경로에서 비교하고, 모델 입력 자체도 확인했습니다. 화면에 포인터가 안 보이거나 결과가 낡는 연결 문제를 고치는 일과, 새 촬영의 눈을 맞히는 일은 다르게 봐야 했습니다.

기존 가로 모델에는 세로 방향의 영상 변화가 수평 출력에도 영향을 주는 경로가 있었습니다. 그렇다고 CNN이 전부 잘못됐다고 단정하지는 않았습니다. 실제로 가로를 2D로 보는 새 후보를 학습해도 촬영을 통째로 뺀 비교에서 기존 모델을 넘지 못했습니다.

저장된 눈 영상은 한 눈당 128×64였지만 원래 카메라에서 잘라 낸 실제 눈 폭은 약 48~95픽셀이었습니다. 저장할 때 확대된 크기를 원본 화질이라고 보고 있지는 않은지 확인해야 했습니다. 128픽셀로 저장했다고 작은 눈 안에 없던 세부 정보가 생기는 것은 아니었습니다.

이후 고해상도 실측을 추가했습니다. 산책 중 의자에서 수집해 불완전했다는 제 설명도 기록에 남겼습니다. 해상도를 높인 회차가 있다는 사실과 해상도만의 개선 효과를 통제해 증명한 것은 다릅니다. 야외·자세·거리·실제 응시가 함께 달라진 자료를 순수한 화질 비교 결과로 쓰지 않았습니다.

## 15 눈가 움직임과 자체 커널 그리고 주시안

### 눈 주위 피부와 근육의 작은 움직임도 단서가 될까

2026년 10월 12일

좌우를 가운데로 오인하는 문제가 남아 눈동자 밖의 단서를 생각했습니다. 눈을 움직일 때 같이 움직이는 눈썹이나 눈 주위 근육이 있다면 그것으로도 방향을 찾을 수 있지 않을까. 저는 피부라는 표현보다 그 아래 움직임을 만드는 근육을 강조했고, 결국 눈 주변의 보이는 움직임부터 비교해 보자고 했습니다.

실험에서는 눈 중심 픽셀을 가리고 바깥 눈가의 모양과 직전 영상 대비 움직임을 따로 추출했습니다. 카메라로 근육 전기 신호를 읽은 것은 아닙니다. 실제로 저장된 영역에는 눈꺼풀과 일부 눈가가 있었지만 눈썹 전체는 없었습니다. 이 자료로 특정 근육의 수축을 측정했다고 쓰면 제 아이디어를 구현보다 크게 포장하는 셈이 됩니다.

눈가의 정적 모양을 더한 후보는 내부 좌우 점수가 71.87%에서 77.67%로 올랐지만 먼 촬영에서는 77.38%에서 57.74%로 떨어졌습니다. 움직임 후보는 야외 점수가 조금 올라도 중립 재현율이 69.00%에서 48.67%로 나빠졌습니다. 가만히 있을 때 원치 않는 이동이 늘 수 있는 결과였습니다.

그래서 더 많은 단서가 있다는 이유만으로 합치지 않았습니다. 눈가의 움직임에는 머리와 카메라의 이동, 매 프레임 달라지는 자르기 위치도 섞일 수 있었습니다. 이후에는 눈 안쪽 움직임과 주변의 공통 움직임을 나눠 보는 연구로 이어졌습니다.

### CNN이 아니라면 눈의 관계를 배우는 자체 커널은 어떨까

2026년 10월 12일

비슷한 실패가 이어지자 CNN을 계속 크게 바꾸는 것만이 답인지 의문이 들었습니다. 눈을 이미지로 통째로 해석하는 방법 외에 홍채·눈 형태·촬영 상태의 수치 관계를 학습하는 방법도 비교하자고 했습니다.

CNN 없는 기하 커널에서는 눈에서 얻은 수치들을 비선형 관계로 펼친 뒤 학습된 계수로 좌·중립·우를 구분했습니다. 커널은 여기서 운영체제의 핵심 부분이나 손으로 정한 판정 문턱이라는 뜻이 아닙니다. 정해 둔 특징을 바탕으로 데이터에서 방향 구분 관계를 배우는 모델입니다. 결정트리 계열도 함께 대조했습니다.

한 비교에서 전체 여섯 촬영 점수는 설치 CNN의 70.03%에서 후보 72.53%로 올랐습니다. 그러나 야외와 반복 촬영의 퇴행 때문에 교체하지 않았습니다. CNN을 빼는 것 자체가 해결책이라는 결론은 얻지 못했습니다.

그다음에는 모델 이름보다 입력을 합치는 방법을 더 살폈습니다. 양쪽 눈을 처음부터 평균해 버리면 차이가 사라질 수 있습니다. 이는 주시안을 따로 보자는 제 다음 가설과 맞물렸습니다.

### 주시안 하나가 답일까 양쪽 눈을 따로 남기면 어떨까

2026년 10월 12일 양안 분리 비교

사람에게 주시안이 있다면 두 눈을 무조건 같은 비중으로 합치는 것이 오히려 방해일 수 있겠다고 생각했습니다. 한쪽 눈만으로 비교하고, 잘 맞는 눈을 선택하는 방식과 자체 커널을 발전시키자는 제안으로 이어졌습니다.

같은 영상 시각의 2,025관측에서 왼눈과 오른눈의 값을 정확히 연결했습니다. 왼눈만, 오른눈만, 두 눈 평균, 두 눈을 따로 유지한 입력, 화질 가중, 학습 자료 안에서만 눈을 고르는 방법을 비교했습니다. 시험 정답을 보고 잘 맞은 눈을 골라 성능처럼 제시하지는 않았습니다.

전체 여섯 촬영의 개발 점수는 왼눈만 58.26%, 오른눈만 62.46%, 평균 76.18%, 양안 분리 77.66%였습니다. 이 점수는 731 평가관측의 촬영·표적 균형 집계이며, 앞서 복구한 2,025관측 전체를 시험했다는 뜻은 아닙니다. 양안 분리는 종전 자체 커널의 68.01%보다 높았습니다. 그러나 먼 거리 점수는 81.55%에서 73.21%로 내려갔고, 중립도 충분하지 않아 제품 승격은 보류했습니다.

따라서 제 주시안을 의학적으로 찾아냈다는 결과는 아닙니다. 현재 자료에서는 한 눈을 버리기보다 두 눈의 차이를 보존한 쪽이 나았다는 관측입니다. 주시안 가설은 양안 정보를 너무 일찍 섞지 말자는 설계 실험으로 발전했습니다.

## 16 중립을 찾는 모델과 실제 출시 운영

### 가운데를 못 찾는 것이 단순한 중립 자료 부족은 아니었다

2026년 10월 13일 중립과 촬영 차이 조사

좌우 방향만 좋아져서는 가운데에서 멈추는 도구가 되지 않았습니다. 촬영을 바꾸면 같은 중앙이 왜 왼쪽이나 오른쪽처럼 보이는지 근본 원인을 다시 확인했습니다. 단순히 중립 예를 더 많이 학습시키면 되는 문제인지도 나눠 봤습니다.

최근 학습 자료에서는 좌·중립·우의 실제 학습 가중치가 이미 비슷했습니다. 그런데 촬영별 중립 입력의 중심 이동폭이 같은 촬영 안의 좌우 변화폭에 비해, 어두운 중심 값은 약 95.2%, 기록된 홍채 평균은 약 112.7%였습니다. 보는 방향의 차이만큼 촬영 기준도 움직일 수 있다는 기술 통계였습니다. 순수 거리 효과나 실제 눈 움직임의 원인 비율을 측정한 것은 아닙니다.

홍채 관측을 따로 쓰기, 눈 안쪽의 명암 분포 보존, 얼굴 문맥에서 공통 중립 중심을 배우기를 비교했습니다. 중립만 좋아지고 작은 눈 조건이 나빠지거나 먼 거리만 좋아지고 다른 방향을 잃는 후보가 있었습니다. 여러 모델의 좋은 숫자만 골라 하나의 성능처럼 합칠 수 없었습니다.

제가 새로 입력한 세 촬영도 승인 후 학습에 사용했습니다. 이 자료를 계속 보호용으로만 두지 말고 모델 개발에도 쓰자는 판단이었습니다. 대신 이제 독립 시험 자료가 아니라고 명시하고, 비교할 때는 시험할 촬영 하나를 통째로 뺐습니다. 같은 방식의 새 촬영 점수는 91.63%에서 93.78%로 올랐지만, 관측 없는 방문까지 실패로 넣으면 후자는 88.37%였습니다. 기존 촬영의 퇴행도 남았습니다.

수치가 높아도 게임에서 잘 안 맞는다는 제 체감이 이상한 것은 아니었습니다. 그 숫자가 어느 창과 어떤 관측을 센 것인지에 따라 뜻이 달랐습니다. 이후에는 정적인 방향 점수와 실제 조작의 안정성·반응을 같은 입력에서 나눠 보는 쪽으로 비교를 옮겼습니다.

### 잘 만든 기능을 사용자가 만나는 곳까지 확인했다

2026년 10월 12일부터 13일 제품 운영

모델 연구와 별개로 Roll·Page·Game의 실제 출시 운영도 이어갔습니다. 앱을 만들고 올리는 것으로 이용자가 생기지는 않았습니다. 첫 사용자가 무엇을 해야 하는지 아는지, 광고와 권한이 시작을 막지 않는지, 스토어와 소개 영상이 쓰임을 보여 주는지도 살펴야 했습니다.

연습은 카메라 권한만으로 가능하게 하고, 다른 앱을 움직이는 접근성 연결과 나눴습니다. 내부 용어 대신 눈을 어떻게 움직이면 화면이 어떻게 바뀌는지 보여 주는 쪽으로 안내를 다듬었습니다. 작은 화면·큰 글자·다른 언어에서 설명이 가려지는 문제도 검사했습니다.

연습 화면에서도 사용자가 시작법을 따로 배워야 하는 구성이 답답했습니다. 저는 전체 화면을 스크롤하는 대신 위아래 조작부를 고정하고 가운데 연습 영역만 바꾸자고 제안했습니다. Roll은 세로로, Page는 가로로 움직이는 콘텐츠에 모드별 눈동작과 결과를 설명하고, 카메라 권한만으로 앱 안에서 먼저 써 본 뒤 다른 앱을 쓰려면 접근성을 켜도록 연결하는 방향이었습니다. 상태 표시가 공간을 과도하게 차지하거나 설명을 가리는 문제도 이 기준으로 다시 봤습니다.

광고에서도 예상하지 못한 차이가 나왔습니다. 영어 제목에서는 괜찮던 영상 영역이 한국어 두 줄 제목에서 약 119dp로 줄어 SDK 경고를 냈습니다. 실제 배치 크기를 확인해 부족하면 광고 대신 설명을 이어 가도록 고쳤고, 수정 Play 설치본의 같은 유형 광고에서 경고가 1개에서 0개로 바뀐 것을 확인했습니다.

시험판과 출시판이 다르게 느껴지는 문제를 줄이기 위해서는 배포 과정도 바꿔야 했습니다. 소스 커밋과 검사 기록을 배포 파일에 연결하고, 같은 Production AAB를 내부 테스트에 올린 뒤 재빌드하지 않고 그대로 승격했습니다. 두 기기에서 실제 Play 설치 버전과 파일 지문, 앱 복귀와 카메라 해제도 확인했습니다.

개발 도중에는 게임의 아직 준비 안 된 Stage 2가 배포판에 보인다는 문제도 지적했습니다. 시험할 수 있다는 것과 공개해도 된다는 것은 달랐습니다. 이후에는 실험 기능을 개발용으로 격리하고, 정식 게임에 Stage 2가 들어가지 않는지도 출시 확인 항목으로 유지했습니다.

광고가 보인다는 것이 수익 성공은 아니고, 같은 파일이라는 확인이 인식 품질을 보장하지도 않습니다. 그래도 어디까지 확인했는지 분명히 하자, 다음 신고가 왔을 때 앱을 잘못 설치한 것인지, 입력이 다른 것인지, 실제 규칙이 잘못된 것인지 나눠 조사할 출발점이 생겼습니다.

# 17 필터와 상태 모델은 체감까지 좋아졌을까

## 깜빡여 켜고 움직인 뒤 다시 깜빡여 끄는 생각

2026년 10월 13일 랩 ON/OFF 구현과 체감 비교

시선이 늘 입력으로 이어져야 하는지도 다시 생각했습니다. 깜빡여 켜고 눈으로 움직인 뒤 다시 깜빡여 끄면, 움직이려 하지 않을 때의 시선까지 조작으로 해석하지 않아도 되지 않을까 하는 상상이었습니다. 마우스를 잡거나 놓는 것과 비슷한 사용 의도 문제였습니다.

이 아이디어는 랩의 별도 ON/OFF 실험으로 구현했습니다. 완전히 감았다 뜨면 조작점 이동이 켜지고 다시 같은 동작을 하면 마지막 위치에 고정됩니다. OFF 중에도 다음 깜빡임을 받기 위해 카메라는 실행합니다. 실험 종료와 앱 이탈은 카메라를 닫습니다. 게임 CNN과 이전·새 커널을 같은 카메라 입력으로 비교하되 게임의 힘 제어나 Unity 보간과는 분리했습니다.

그 과정에서 눈을 뜬 첫 프레임이 OFF 명령보다 먼저 도착해 고정 직전에 점이 튀는 순서 버그도 나왔습니다. 눈을 감는 후보가 시작되면 위치를 유지하고, 공통 문법의 완료 사건으로 ON/OFF를 한 번만 바꾼 뒤 새 열린 눈 관측부터 사용하도록 고쳤습니다. 늦게 온 이전 실행의 결과가 다시 움직이지 않게 세대와 사건 번호도 구분했습니다.

구현 경계와 카메라 수명, 인공 문법으로 연결을 검사했지만 실제 눈으로 토글한 성공률까지 그 검사로 증명한 것은 아닙니다. 완성 파형이 같으면 자연 깜빡임도 토글될 수 있다는 한계도 남았습니다. 켜고 끄는 조작 의도를 나눈 것과 방향 모델의 정확도를 올린 일은 달랐습니다.

이때 표시되는 좌·중립·우의 세 위치는 실제 화면의 세 좌표를 측정한 점이 아니었습니다. 분류 결과를 보기 좋게 배치한 것입니다. OFF로 멈춘 모습을 정확도 개선으로 세지 않았고, 랩의 사용법을 모든 제품 사용자의 필수 절차로 바꾸지도 않았습니다.

## 작은 화면의 떨림을 줄이면 해결되지 않을까

2026년 10월 13일 필터 비교와 이후 체감

가까이 보면 좌·중립·우의 왕복이 조금 줄어드는 경험에서 화면 크기와 거리의 관계를 생각했습니다. 한 점을 본다고 해도 화면 전체가 시야에 들어오고 눈에는 작은 움직임도 있을 수 있습니다. 작은 방향 차이가 잡음에 묻히는 것은 아닐까 하는 가정에서 이동평균, EMA, One Euro를 비교하자고 제안했습니다.

작은 범위의 움직임은 입력으로 보지 않는 데드존도 함께 제안했습니다. 다만 현재 문서에 연결한 네 방식 비교는 원본·이동평균·EMA·One Euro이며, 별도 데드존 후보까지 검증해 해결했다고 쓰지는 않습니다. 작은 유효 방향 변화도 함께 지워 버릴 수 있다는 점은 추가로 확인할 질문으로 남겼습니다.

이 가설을 눈의 미세 운동이 원인이라고 확정하지는 않았습니다. 현재 영상에는 실제 안구 운동, 얼굴 움직임, 랜드마크 오차가 섞여 있고 각각을 독립적으로 측정하지 않았기 때문입니다. 우선 같은 모델 점수를 한 번만 계산해 네 방식에 넣고, 흔들림과 추가 지연, 새 미실행을 함께 비교했습니다.

평가 일곱 촬영에서 이동평균은 판정 전환을 약 50.67% 줄였지만 추가 지연 p95가 294.4ms였고 원본 성공 중 세 방문이 새로 실패했습니다. One Euro는 전환을 약 14.22% 줄이고 추가 지연 p95가 59.6ms였지만 별도 촬영의 오른쪽 재현율이 0.48%p 내려갔습니다. 흔들림만 보고 채택할 수는 없었습니다.

제 체험 후기는 별차이가 없다는 것이었습니다. 이 의견을 필터 수치를 더 강하게 조정하라는 주문으로만 받으면 같은 실험을 반복할 수 있습니다. 이미 다른 곳을 보고 있다고 판단한 점수를 부드럽게 만드는 것에는 한계가 있습니다. 그래서 다음에는 출력 평균이 아니라 눈의 상대 움직임과 상태를 배우는 쪽으로 질문을 바꿨습니다.

### 덜 왔다 갔다 해도 틀린 방향에 오래 머물 수 있었다

2026년 10월 15일 상대 움직임과 상태 모델

눈가 전체가 함께 움직이는 부분과 눈 안쪽의 움직임을 나눠 보고, 현재 방향을 판단하는 모델과 이전 상태에서 어떻게 바뀌는지를 배우는 모델도 분리했습니다. 다른 분야에서 같은 현상을 어떻게 다루는지 찾아보자는 제 요청에 따라, 시간에 따른 상태 전이 같은 개념을 참고하되 실제 구현과 원 논문의 차이도 남겼습니다.

같은 기록의 한 후보는 적격 일치도가 82.01%에서 86.04%로 오르고 판정 변경이 82회에서 15회로 줄었습니다. 여기까지만 쓰면 상당히 좋아 보입니다. 그러나 오래 지속된 오판은 12.039초에서 15.552초로 늘었고, 추가 지연 p95는 206.8ms였습니다. 새 미실행도 남았습니다.

즉 잘못된 방향도 덜 흔들리게 된 경우가 있었습니다. 현재 판단이 틀렸는데 이전 기억까지 같은 답을 지지하면 자신 있게 오래 틀릴 수 있습니다. 이 비교만으로 그 기전까지 확정한 것은 아니지만, 안정성과 정확도만이 아니라 틀린 상태에서 회복하는 능력도 따로 봐야 한다는 결과였습니다.

그래서 이 후보를 게임의 기본 모델로 승격하지 않았습니다. 여기의 일치도는 안내된 표적과 저장 관측을 비교한 단일 사용자 개발 지표이며, 새로운 사람의 실제 시선 좌표를 그 비율로 맞힌 성적이 아닙니다. 더 높은 숫자가 나와도 사용자의 다음 입력을 놓치면 더 좋은 도구라고 말할 수 없었습니다.

## 18 처음의 필요와 끝내 남은 문제

### 아이디어를 많이 낸 것보다 무엇이 달라졌는지 남기기

2026년 10월 15일 정리 기준

시선 연구의 흐름을 한 줄로 줄이면 많은 모델을 바꿨다는 말이 됩니다. 하지만 실제 과정은 달랐습니다. 점이 튀어서 보는 힘을 생각했고, 아래가 안 보여 눈꺼풀을 생각했으며, 촬영을 바꾸면 틀려 얼굴 각도와 거리를 모으기 시작했습니다. 한쪽 눈을 따로 보자는 생각은 양안 분리로, 눈 주위 근육을 보자는 생각은 주변과 내부의 움직임을 나누는 실험으로 이어졌습니다.

그 과정에서 일부 조건의 방향 구분, 표본 수집과 시간 연결, 입력·제어·화면 표시를 나눈 구조는 발전했습니다. 반면 개인 보정 없이 촬영을 바꿔도 자연스럽게 작동하는 공통 시선 좌표, 흔들림·지연·지속 오판을 동시에 줄이는 모델은 아직 완성하지 못했습니다. 결과가 남지 않은 초기 가설을 성공이나 실패로 임의 정리하지도 않았습니다.

제가 맡은 핵심은 불편함을 구체적으로 제기하고, 기존 가정이 맞는지 되묻고, 새 가설과 성공 범위를 정하며 직접 체감 결과로 채택을 판단하는 일이었습니다. 구현·학습·반복 검증과 문서화에는 AI 코딩 도구를 활용했습니다. 제가 제안한 아이디어와 외부 기술, 함께 만든 구현을 구분해 쓰는 것이 이 경험을 정확히 설명하는 방법이라고 생각합니다.

### 출시와 모델 완성을 섞지 않기

제품과 연구의 구분

Roll·Page·Game은 출시했지만 시선 기능까지 완성했다고 쓰지는 않습니다. 구형 기기의 일부 오래 감기 실패와 촬영이 바뀔 때의 시선 판단은 남은 과제입니다. 특허도 검토와 제출 자료 준비를 진행한 경험으로 구분합니다. 이 프로젝트에서 얻은 경험은 기능을 늘린 목록보다, 불편함을 발견할 때마다 문제의 정의와 설계를 다시 바꿔 온 과정에 있습니다.

## 19 기술 선택과 확인한 한계

서로 다른 실험의 장점을 한 모델의 성능으로 합치지 않습니다. 다음 표는 선택 이유와 남은 범위를 나란히 정리한 것입니다.

| 구체적인 문제 | 선택한 방법 | 확인 범위와 한계 |
| --- | --- | --- |
| 자연 깜빡임과 의도 입력이 비슷함 | 완성 파형과 유지·재열림 관계를 읽는 문법 | 기기·사용자 차이의 일부 실패는 남음 |
| 표시가 감김인데 화면이 안 움직임 | 관측·문법·실행 허용·접근성 접수를 따로 추적 | 표시 성공을 종단 성공으로 세지 않음 |
| 앱과 시험판마다 의미가 달라짐 | 공통 의미 해석과 제품·빌드 어댑터 분리 | 파일 공유만으로 경로 일치를 가정하지 않음 |
| 시선 좌표가 들쭉날쭉함 | 보는 힘과 탄성 추종 비교 | 안정화와 원시 좌표 정확도는 별개 |
| 눈동자를 찾기 어려움 | 흰자·밝기·경계와 눈꺼풀 가설 비교 | 흰자 정답 분할·깜빡임 구별 완성 아님 |
| 거리와 자세가 달라짐 | 상대 눈 크기·폰 자세·얼굴 자세 추정값을 구분 | 상대 크기는 cm 단위 거리, 얼굴 각도는 물리 정답이 아님 |
| 작은 눈만으로 단서가 부족함 | 얼굴 먼저 추정 후 눈으로 수정 | 조건별 이득과 퇴행이 공존해 미채택 |
| 양쪽 눈을 섞으면 차이가 사라짐 | 한쪽 눈·평균·양안 분리 커널 비교 | 의학적 주시안 판정 아님 |
| 중립 기준이 촬영마다 달라짐 | 촬영 전체 제외 평가와 중립 입력 진단 | 촬영별 분포 이동은 확인, 물리적 원인 미확정 |
| 필터가 정상 입력도 늦춤 | 흔들림·추가 지연·새 미실행·지속 오판 동시 평가 | 평균이 높아도 품질 관문 실패면 보류 |
| 다른 앱과 함께 쓰면 버벅임 | 대기 작업 예산과 Rust 동일 결과 변환 | 특정 연산 개선이지 전체 발열 해결 아님 |
| 출시 후 시험판과 다르게 느껴짐 | 동일 AAB 승격과 기기 설치 지문 확인 | 같은 파일이어도 입력 조건의 차이는 남음 |

# 20 프로젝트와 연구 계보

| 시기 | 갈라진 질문 | 남은 결과 |
|---|---|---|
| 2026년 6월 | 브라우저 깜빡임 실험 | 모바일 HTTPS·카메라 준비와 당시 미해결 깜빡임→스크롤 연결을 구분 |
| 2026년 7월 | 눈 입력 문법과 특허 준비 | 시간 하나 대신 파형 관계, 뜻과 실행 분리, 출원 검토·자료 준비 |
| 2026년 7월 말부터 8월 | Roll·Page·Game으로 확장 | 공통 입력과 제품별 행동, 앱별 수집·실행 경계 정리 |
| 2026년 8월 | 전체 화면과 보는 힘 | 테두리 보정·전체 격자·직접 좌표와 제어 비교 |
| 2026년 8월 말부터 9월 | Line과 시선 문맥 | 줄 위치·전달 지연·폰 자세·화면 안팎 실험 분리 |
| 2026년 10월 8일 | 관측과 단서 재검토 | 얼굴 자세 추정값 누락 수정, 얼굴 우선·눈꺼풀·오차 분해 |
| 2026년 10월 9일부터 13일 | 좌우·상하와 공통 모델 | 목표 축소, 눈가·자체 커널·양안 분리·중립·촬영 전이 비교 |
| 2026년 10월 13일부터 15일 | 시간축과 체감 검증 | 원본·필터·상태 모델의 안정성·지연·미실행·지속 오판 비교 |

# 21 이력서에 옮길 때 구분할 표현

이 원고는 과정을 자세히 설명하는 첨부 자료입니다. 이력서 본문에서는 지원 직무와 가까운 사례를 고르되, 구현·실행·개발 지표·제품 정확도를 같은 성과로 바꾸지 않습니다.

| 피할 표현 | 정확한 표현 |
|---|---|
| 시선 정확도 93.78%를 달성했다 | 특정 세 촬영의 조건부 개발 점수다. 관측 없는 방문 포함 시 88.37%이며 새 사용자 정확도가 아니다. |
| 흰자를 인식해 시선 추적을 완성했다 | 흰자 변화 가설과 명암·경계 입력을 비교했다. 정답 마스크를 갖춘 흰자 분할 성공은 미확인이다. |
| 눈 근육의 수축을 측정했다 | 눈가의 영상 모양과 움직임을 시험했다. EMG나 특정 근육 활성 측정은 아니다. |
| 사용자의 주시안을 자동으로 찾았다 | 한쪽 눈과 양안 결합 방식을 비교했다. 의학적 주시안은 확정하지 않았다. |
| 필터로 흔들림 문제를 해결했다 | 일부 흔들림 감소와 지연·오판의 손실을 함께 확인했으며 자동 채택하지 않았다. |
| Rust로 앱이 61.9% 빨라졌다 | SM-S의 특정 영상 변환 계산 시간이 약 61.9% 감소했다. 전체 앱 속도나 발열 개선율이 아니다. |
| 눈감김 문법 특허를 출원·등록했다 | 선행기술 검토, 명세서와 제출 후보 준비까지 확인했다. 접수·등록 완료 증거는 확인하지 않았다. |
| 시선 기능을 게임 정식판에 완성했다 | 실험과 디버그 적용을 분리했고 준비되지 않은 Stage 2는 프로덕션 공개 범위에서 제외했다. |

# 22 확인한 원본과 공개 범위

아이디어의 출발과 체감은 작성자의 실제 대화와 시험 설명을 바탕으로 재구성했습니다. 정량 결과는 날짜별 실험 문서를 인용했습니다. 대화에 없는 직접 인용이나 감정을 만들지 않았습니다. 미확인 제안일은 표시했고, 병행한 가지를 전부 순차적인 성공으로 묶지 않았습니다.

원본 제품 저장소와 촬영 코퍼스는 비공개입니다. 공개 저장소에는 다시 쓴 서사, 집계와 원천 문서 식별 해시만 넣었습니다. 그 해시는 과거 원천을 식별하는 값이지 독립 재현이나 수치 진실성을 외부에서 증명하는 도구가 아닙니다.

실제 눈·얼굴 이미지, 영상, 특징 행, 모델 가중치, 기기·계정 식별자와 미출원 명세서·청구항·도면은 공개하지 않았습니다. 특허 관련 표현은 검토·준비 범위이며 실제 접수나 권리 확보를 뜻하지 않습니다.

공개 개발 사례와 최신 정정 기록

실험 수치의 평가 조건과 원천 식별 목록

아이디어 누락과 표현 재검토 기록

주요 기록: 시선 연구 초기 계보, GazeLab, BlinkLine, 얼굴 각도·얼굴 우선·눈꺼풀·오차 분해, 3×3 축 분리, 눈가 움직임·기하 커널·양안 분리, 중립·필터·상태 모델, 깜빡임 문법·문제 해결 원장과 출시 이력. 전체 식별 목록은 공개 저장소의 evidence/selected-results.json에 있습니다.